

CSE 425: Concepts of Programming Languages

A brief

Introduction to Fortran

A decorative graphic consisting of four horizontal lines. The top and bottom lines are thin and grey. The two lines in the middle are thicker and red, with the top red line being slightly longer than the bottom red line.

Md. Shahriar Karim

Assistant Professor

Department of Electrical and Computer Engineering

North South University

Bangladesh

Fortran95: CodeBlocks: Steps



Code::Blocks

The open source, cross-platform IDE

[Release 17.12 rev 11256 \(2017-12-28 10:44:41\) gcc 5.1.0 Windows/unicode - 32 bit](#)

 [Create a new project](#)  [Open an existing project](#)  [Tip of the Day](#)

 [Visit the Code::Blocks forums](#) [Report a bug or request a new feature](#)

New from template

Projects
Build targets
Files
Custom
User templates

Category: <All categories>

 ARM Project	 AVR Project	 Arduino Project	 Code::Blocks plugin
 Console application	 D application	 Direct/X project	 Dynamic Link Library
 Empty project	 FLTK project	 Fortran DLL	 Fortran application
 LTI	 GLFW	 GLUT	

Compiler settings

Global compiler settings

 Global compiler settings

 Profiler settings

Selected compiler: GNU Fortran Compiler

[Set as default](#) [Copy](#) [Rename](#) [Delete](#) [Reset defaults](#)

Compiler settings | Linker settings | Search directories | Toolchain executables | Custom variables | Build options

Policy:

Compiler Flags | Other compiler options | Other resource compiler options | #defines

☒ **Debugging**

Produce debugging symbols [-g] ☐

☒ **Run-time checks**

Good programming style

- The logical structure of the program should be as easy as possible
 - Use comments for terms that are not self-explanatory. Also, avoid comments for obvious computational expressions

Similarly if you have a loop, a comment of the form below is of no help:

```
! loop from 1 to 10  
do i=1,10
```

➤ But a comment of the following form, say in a program calculating a binomial might be very useful:

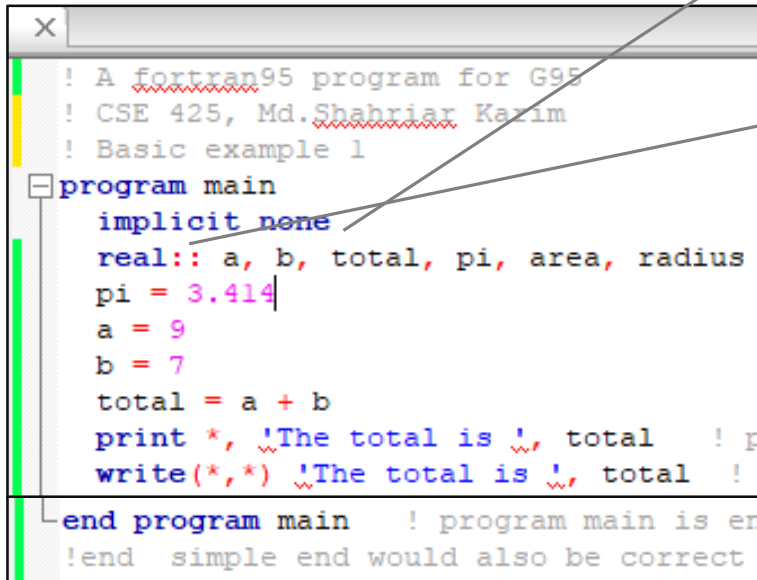
```
! loop to calculate  $nCr$   
do k=1,r
```

Image source: Dept. of Physics, University of Cambridge

- Indenting of the code blocks is highly appreciated, as it tends to improve the readability

Declarations/initialization of variables

Declarations:



```
! A fortran95 program for G95
! CSE 425, Md.Shahriar Razim
! Basic example 1
program main
  implicit none
  real:: a, b, total, pi, area, radius
  pi = 3.414
  a = 9
  b = 7
  total = a + b
  print *, 'The total is ', total
  write(*,*) 'The total is ', total
end program main
!end simple end would also be correct
```

- Turns off the **implicit type definition** and default naming convention initial Fortran version allowed
- **:: is the separator**,
 - necessary for type specification statement where the variable values are initialized
 - But is also used to define variable type without initializing the variable. For instance, both **real :: a, b** and **real a, b** are valid definition

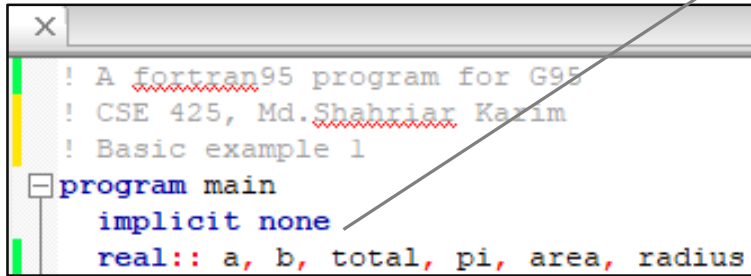
Precision:

- **KIND**: This type parameter (KIND) is used to define the precision of a real, integer, complex, or logical variable.
- For example, a real number with at least 10 decimal places of precision with a range of at least -10^{34} to 10^{34} can be defined as:

REAL(KIND = SELECTED_REAL_KIND(10,34)) :: VARIABLE_NAME

Data Types and Variables

Declarations:



```
! A fortran95 program for G95
! CSE 425, Md.Shahriar Karim
! Basic example 1
program main
  implicit none
  real :: a, b, total, pi, area, radius
```

```
TYPE :: VARIABLE NAME
TYPE VARIABLE NAME
```

- Turns off the implicit type definition and default naming convention initial Fortran version allowed

- Variable names starting with i, j, k, l, m, or n, are integers.
- Variables starting with other alphabets are real variables.

Data Types: ▪ Character ▪ Real ▪ Integer ▪ Complex ▪ Logical

```
character (len = 20) :: test
test = 'Hello World'
```

```
integer :: number1
Integer :: number2
```

Constant:

- Definition of constant just require a 'parameter' right after the type of the variable.

```
integer, parameter :: constant1
real, parameter :: pi = 3.1428
```

Conventions: variables and operators

- **Fortran77:** Variable name allowed 1-6 characters length chosen from letters a-to-z and digits 1-to-10
- **Fortran90:** Variable name allowed 1-31 characters length chosen from letters a-to-z and digits 1-to-10. It also allows (`_`) underscore in variable names.
 - `abc` valid variable name; `ABC` denotes the same. So, it is not case sensitive
 - `123` invalid variable name; also, `123abc` is invalid. That is, variable name must start with a character.
- **Operators**

Operator	Precedence	Meaning
<code>**</code>	1	Raise to the power of
<code>*</code>	2	Multiplication
<code>/</code>	2	Division
<code>+</code>	3	Addition or unary plus
<code>-</code>	3	Subtraction or unary minus

You can change the precedence by using brackets; sub-expressions within brackets are evaluated first.

Program skeleton

- Start with `program main`, and it is enclosed within `end program main`. Within the enclosure, codes are `indented (recommended)`.
- Data type needs to be defined: `real :: a, b`
- Not case sensitive: does not create error if `total` is used later as `Total`

```
*main.f95 x
1  ! A fortran95 program for G95
2  ! CSE 425, Md.Shahriar Karim
3  ! Basic example 1
4  program main
5      implicit none
6      real:: a, b, total, pi, area, radius
7      pi = 3.14
8      a = 9
9      b = 7
10     total = a + b
11     print *, 'The total is ', total    ! printing output
12     write(*,*) 'The total is ', total ! printing output
13
14     ! calculate the area of the circle
15     radius = 4;
16     area = pi*radius**2    ! ** symbol for square of the quantity
17     print *
18     print *, 'The area is ', area    ! printing output
19
20     write(*,*) 'Radius was ', radius ! printing output
21
22 end program main    ! program main is ended here
23 !end simple end would also be correct
```

Fortran Program: Input/output

- **read:** Used to take input from the external source.
- **write:** Used to outputting information from the program.

The form of the I/O statements is as follows:

```
read(stream, label [, end=end] [, err=err]) list  
and  
write(stream, label) list
```

- **stream:** number linked to previously defined file, a character; or, * can be used to indicate default value (screen of a terminal session). If stream is a character variable, write command stores the value in that variable.
- **label:** It is the line number where the **format** statement. However, it can be replaced using * to use free-format.
- **list:** list of items (separated by commas) to be transferred to the output window; it can also include quoted text strings

Logical controls: if else and nested if else

■ Syntax

```
if (Condition) then
statement1
else if (Condition) then
Statement2
else
statement3
end if
```

```
if (Condition) then
Statement1
else
Statement2
end if
```

.lt. or	<	less than
.le. or	<=	less than or equal
.eq. or	==	equal
.ge. or	>=	greater than or equal
.gt. or	>	greater than
.ne. or	/=	not equal

.not.	not
.and.	and
.or.	inclusive or

Image source: Dept. of Physics, University of Cambridge

```
main.f95 x
1  ! A fortran95 program for G95
2  ! CSE 425, Basic Example 3
3  program lettergrade
4      implicit none
5      real :: score
6      print *, 'Enter CSE 425 score'
7      read *, score
8      if (score>=93) then
9          write(*,*) "Letter grade is A"
10     else if (score>=90 .and. score <93) then
11         write(*,*) "Letter grade is A-"
12     else if (score>=87 .and. score <90) then
13         write(*,*) "Letter grade is B+"
14     else if (score < 87) then
15         write(*,*) "Letter grade is B"
16     end if
17 end
```

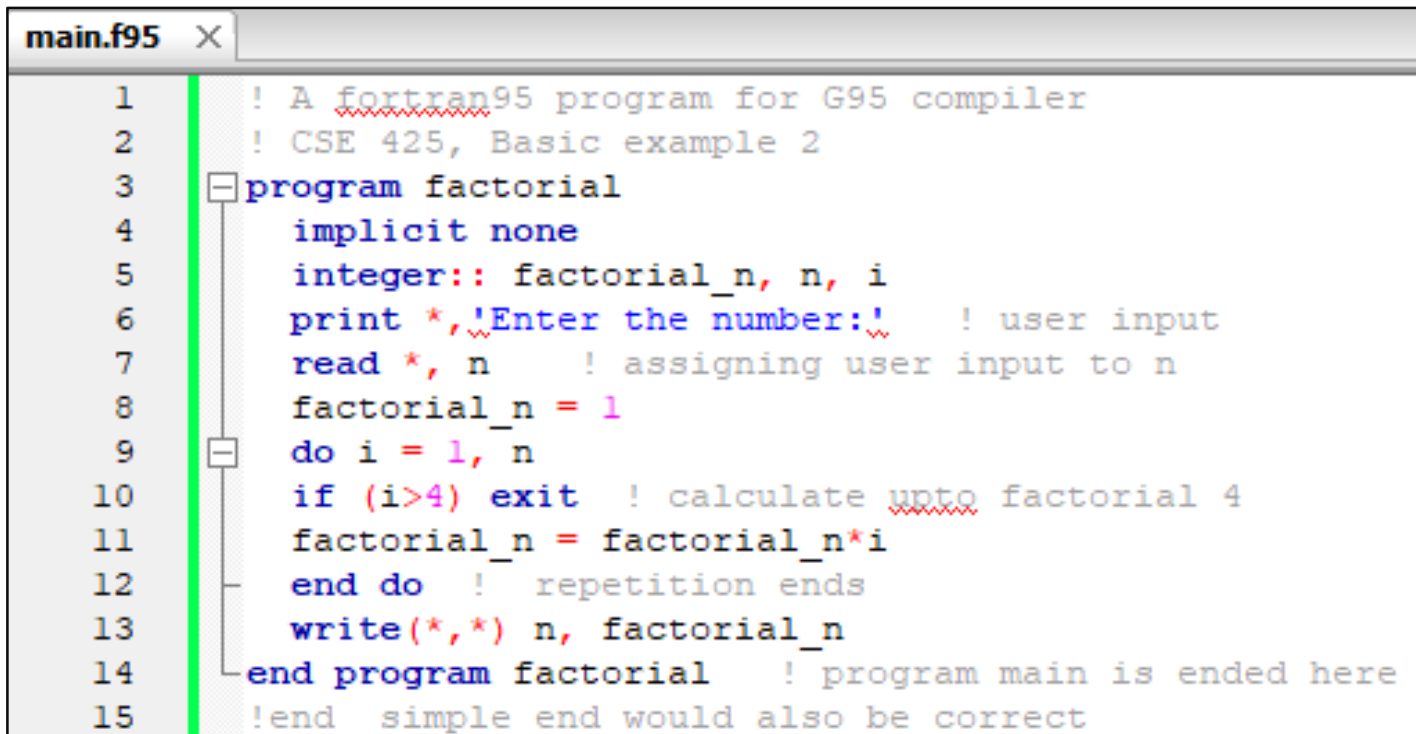
Repetition: do loop

- Syntax

```
do variable = start_range, stop_range [,step]
statements
end do
```

- The `exit` command can terminate the loop

Factorial calculation



The screenshot shows a code editor window titled 'main.f95'. The code is a Fortran 95 program for calculating factorials. It includes comments in red and code in blue. A green vertical line is positioned at the start of line 9. A bracket on the left side of the code block groups lines 9 through 13, indicating the loop structure.

```
1  ! A fortran95 program for G95 compiler
2  ! CSE 425, Basic example 2
3  program factorial
4      implicit none
5      integer:: factorial_n, n, i
6      print *, 'Enter the number:!'    ! user input
7      read *, n    ! assigning user input to n
8      factorial_n = 1
9      do i = 1, n
10         if (i>4) exit ! calculate upto factorial 4
11         factorial_n = factorial_n*i
12     end do ! repetition ends
13     write(*,*) n, factorial_n
14 end program factorial ! program main is ended here
15 !end simple end would also be correct
```

Inner-Outer: Logical Controls

- In large program, logical control could be titled as `outer` and `inner` to improve the readability of the program
- **Example:** Finding the `square root` of a `real number` and dividing it further by another `real number`. Also, the `divided by zero` problem has been discarded.

```
main.f95 x
1      ! A fortran95 program for G95
2      ! CSE 425, Basic Example 4
3      program sqrt_calculation
4      implicit none
5      real :: desired_no, divisor_no, x
6      write(*,*) 'Find the square root of a number/divisor!'
7      print *
8      write(*,*) 'Enter the divisor and target number!'
9      read *, divisor_no, desired_no
10     outer: if (divisor_no /= 0) then
11         inner: if (desired_no < 0) then
12             write(*,*) 'Invalid input!'
13         else inner
14             x = sqrt(desired_no)/divisor_no
15             write(*,*) 'Divisor number is', divisor_no
16             write(*,*) 'Other number is', desired_no
17             write(*,*) 'sqrt(desired_no)/divisor_no =', x
18         end if inner
19     else outer
20
21         write(*,*) 'Divided by zero issue!'
22     end if outer
23
24     end program sqrt_calculation
25
```

Intrinsic functions for computation

- FORTRAN provided a list of intrinsic functions that are frequently used for scientific computation.

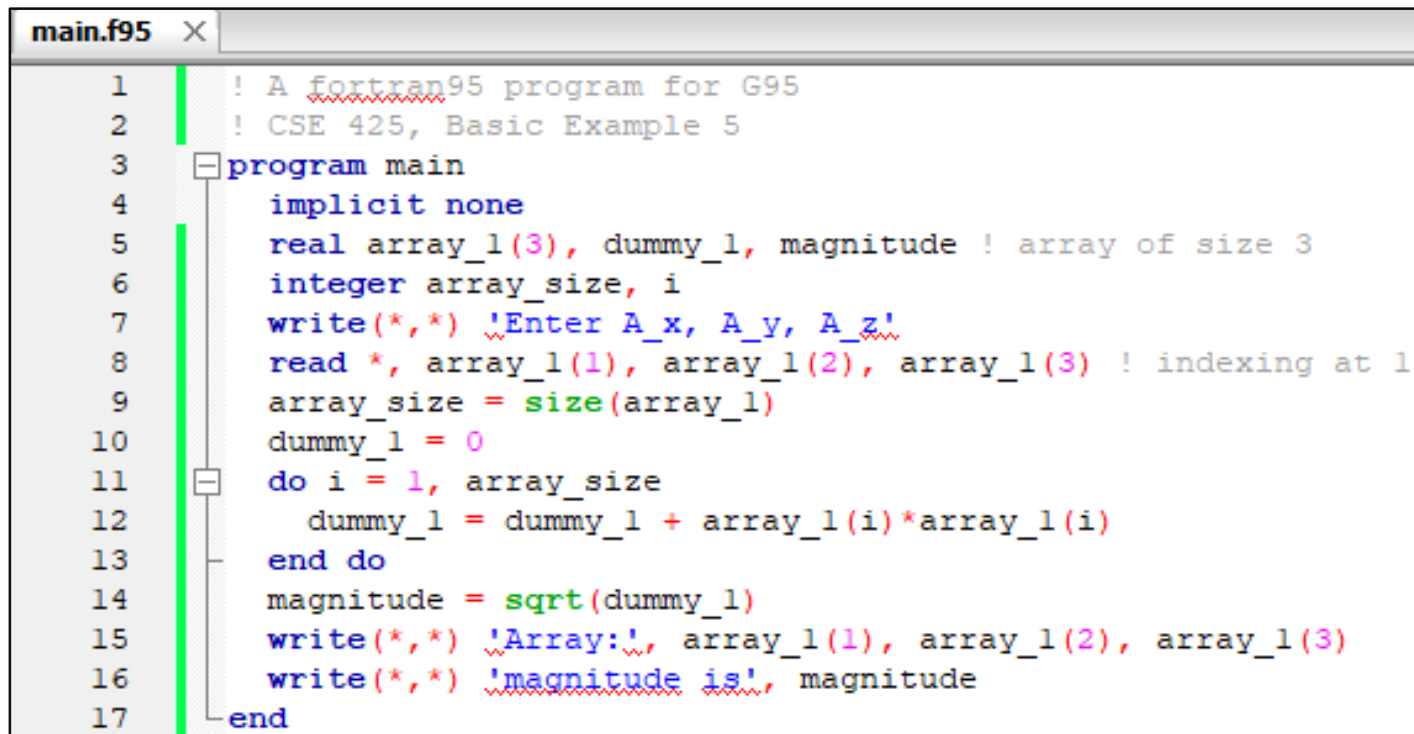
Name	Action
ABS (A)	absolute value of any A
ACOS (X)	inverse cosine in the range $(0,\pi)$ in radians
AIMAG (Z)	imaginary part of Z
AINIT (X [,KIND])	truncates fractional part towards zero, returning real
ANINT (X [,KIND])	nearest integer, returning real
ASIN (X)	inverse sine in the range $(-\pi/2,\pi/2)$ in radians
ATAN (X)	inverse tangent in the range $(-\pi/2,\pi/2)$ in radians
ATAN2 (Y, X)	inverse tangent of Y/X in the range $(-\pi,\pi)$ in radians
CMPLX (X [,Y] [,KIND])	converts to complex $X+iY$; if Y is absent, 0 is used
CONJG (Z)	complex conjugate of Z
COS (W)	cosine of argument in radians
COSH (X)	hyperbolic cosine
EXP (W)	exponential function
FLOOR (X)	greatest integer less than X

A few more examples

```
*main.f95 X
1  ! A fortran95 program for G95
2  ! Complex Number
3  program complex_number
4  implicit none
5  ! Define variables and constants
6  complex, parameter :: i = (0, 1) ! sqrt(-1)
7  complex :: num1, num2
8  num1 = (3, 4); num2 = (3, -4)
9  write(*,*) i * num1 * num2
10 end program complex_number
```

Array in Fortran

- Array of variables is set up in the declaration set up; array indexing depends on how the declaration is done
- Syntax:
`real :: array_1(3) ! Array of 3 values`
`real :: array_2(3, 3) ! Rank is 2 and it is a matrix`
- Example



```
main.f95 x
1  ! A fortran95 program for G95
2  ! CSE 425, Basic Example 5
3  program main
4      implicit none
5      real array_1(3), dummy_1, magnitude ! array of size 3
6      integer array_size, i
7      write(*,*) 'Enter A_x, A_y, A_z'
8      read *, array_1(1), array_1(2), array_1(3) ! indexing at 1
9      array_size = size(array_1)
10     dummy_1 = 0
11     do i = 1, array_size
12         dummy_1 = dummy_1 + array_1(i)*array_1(i)
13     end do
14     magnitude = sqrt(dummy_1)
15     write(*,*) 'Array:', array_1(1), array_1(2), array_1(3)
16     write(*,*) 'magnitude is', magnitude
17 end
```

Varying Array in Fortran

- Array size can be left open, and it can be assigned as needed. Also, such array could be released free once the task is completed.

- Syntax:

```
real, dimension(:), allocatable :: array_1 !  
We define an array of unknown size array_1
```

- The array size later could be fixed as follows:

```
s = 100 ! array size  
allocate(array_1(s))
```

- After the task is finished, the array is deallocated as follows:

```
s = 100  
deallocate(array_1)
```

Varying Array in Fortran

- Mathematical computations can be performed directly on the array itself.

```
real :: array_a(5), array_b(5), array_c(5)
integer :: I

do i = 1, 10
  array_c(i) = array_a(i) + array_b(i)
end do
```

↓ Instead

```
real :: array_a(5), array_b(5), array_c(5)
array_c = array_a + array_b
```

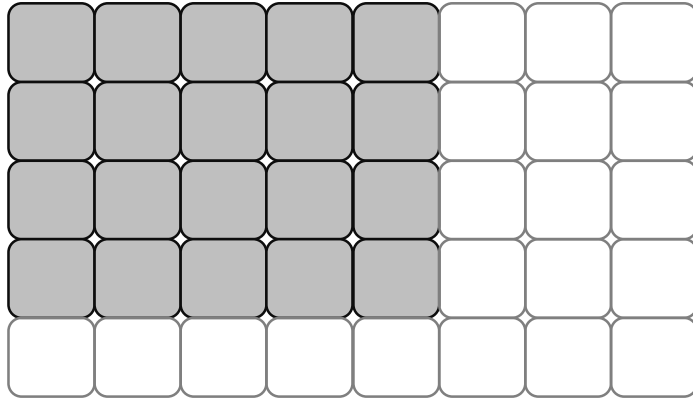
Conditional `where: in array`

- Syntax

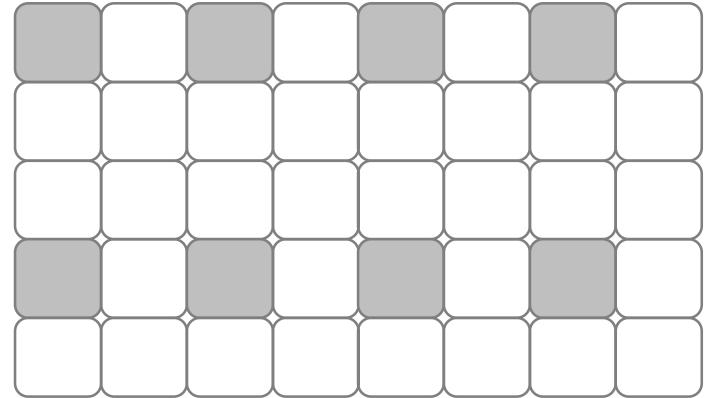
```
where (logical array statements)
      statements
elsewhere
      alternative statements
end where
```

Array structure

```
real, dimension(1:4,1:5) :: array_1
```



```
real, dimension(1:6:3,1:7:2) :: array_1
```



Dynamic Array

```
main.f95 x
1  ! A fortran95 program for G95
2  ! Basic Example: Dynamic Array
3  program dynamic_array_example
4      implicit none
5      real, dimension(:,,:), allocatable :: dynamic_1
6      integer :: row_dim, column_dim
7      integer :: i, j
8
9      print *, 'Enter row_dim and column_dim!'
10     read *, row_dim, column_dim
11
12     allocate(dynamic_1(row_dim,column_dim))
13     do i = 1, column_dim
14         do j = 1, row_dim
15             dynamic_1(j,i) = i*j
16             print *, "dynamic_1(",j,",",i,") = ", dynamic_1(j,i)
17         end do
18     end do
19     !deallocate(dynamic_1)
20 end program
```

Mandatory syntax:
dynamic_1
becomes dynamic

Allocating
necessary memory
location as per the
need

Multiplication of Vector $\mathbf{v}$ and Identity Matrix $\mathbf{I}$

```
integer name
ii=name(x,y,z)
stop
end
```

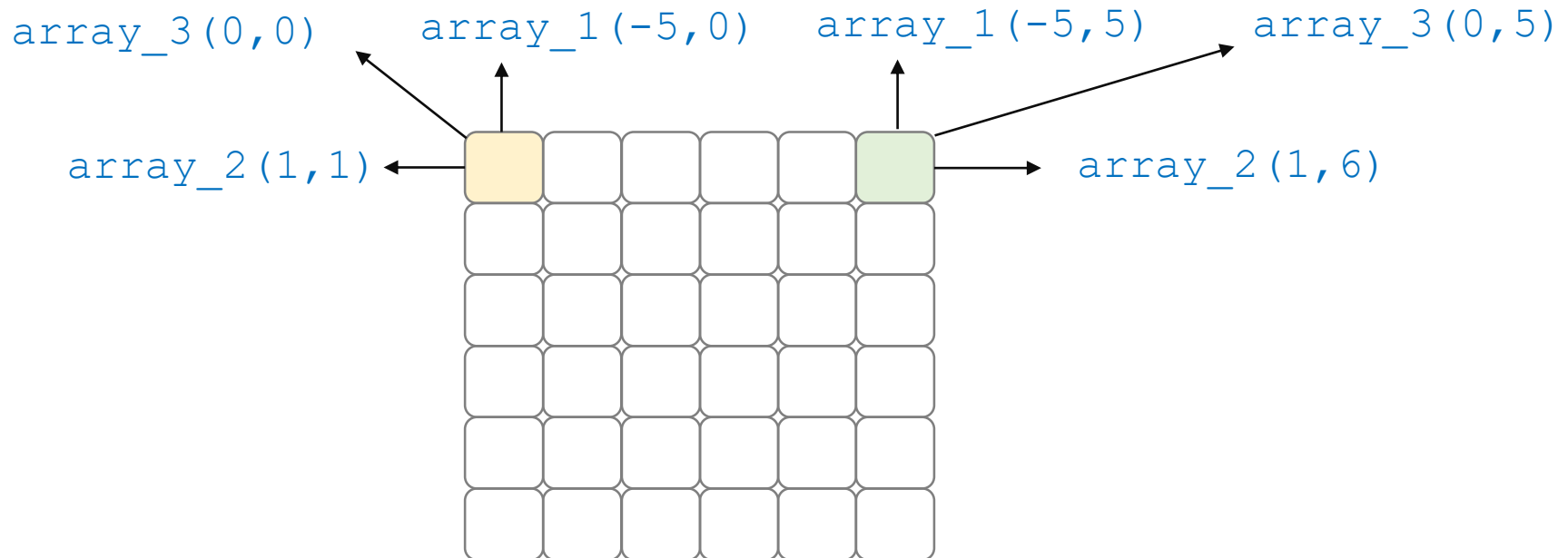
```
function name(x1,y1,z1)
integer name
name=int(x1+y1+z1)
return
end
```

```
main.f95 x
1      ! A fortran95 program for G95
2      ! Basic Array Example
3      program main
4          implicit none
5          real :: vec_1(3), vec_2(3), mtx_I(3,3)
6          integer :: i, j
7          vec_1(1) = 5
8          vec_1(2) = 5
9          vec_1(3) = 5
10         ! matrix initialization I
11         do i = 1, 3
12             do j = 1, 3
13                 if(i>j .or. i<j) then
14                     mtx_I(i,j) = 0
15                 else
16                     mtx_I(i,j) = 1
17                 end if
18             end do
19         end do
20         ! matrix-vector product
21         do i = 1, 3
22             vec_2(i) = 0
23             do j = 1,3
24                 vec_2(i) = vec_2(i) + mtx_I(i,j)*vec_1(j)
25             end do
26         end do
27
28         write(*,*) 'vec_1 =',vec_1
29         write(*,*) 'matrix vector product: AI', vec_2
30         write(*,*) ' matrix is: ', mtx_I
31     end
```

Conditional `where: in array`

- Negative indexing is allowed. Also, user can define the starting and ending indexes as needed.

```
real, dimension(-5:0,0:5) :: array_1  
real, dimension(6,6) :: array_2  
real, dimension(0:5,0:5) :: array_3
```



Functions

- Functions are generally placed after the `end` part of the main program.
- Function definition starts with the declaration of the `type of value` the defined function is expected to return. However, `type of the value` could be defined within the function definition.
- It also include the `function name`, and the `argument list` it takes as the inputs.
- All the `variables that are used` in the function, including `the arguments` of the function, must have `type declaration` in the function immediately after `the first line` of the function.
- The `function name` must be used `in an assignment statement` within the defined function.
- The defined function must finish with `end` statements

Functions: External Definition

- Calculation of the average of three input numbers using **external function** definition

```
main.f95 x
1      ! A fortran95 program for G95
2      ! Basic Example: Function Definition
3      program main
4          implicit none
5          real avg, a, b, c, cal_avg
6          write(*,*) 'Enter three numbers'
7          read *, a, b, c
8
9          avg = cal_avg(a, b, c)
10         write(*,*) 'The three numbers', a, b, c
11         write(*,*) 'Average is ', avg
12
13     end program main
14
15     real function cal_avg(x, y, z)
16         real x, y, z, sum
17         sum = (x + y + z)
18         cal_avg = sum/3
19         return !obsolete in f90 & f95
20     end function cal_avg
```

External function
definition

Example: Function Definition

```
main.f95 x
1  ! A fortran95 program for G95
2  ! Basic Example: Function Definition
3  program main
4      implicit none
5      real avg, a, b, c, cal_avg
6      write(*,*) 'Enter three numbers'
7      read *, a, b, c
8
9      avg = cal_avg(a, b, c)
10     write(*,*) 'The three numbers', a, b, c
11     write(*,*) 'Average is ', avg
12
13 end program main
14
15 real function cal_avg(x, y, z)
16     real x, y, z, sum
17     sum = (x + y + z)
18     cal_avg = sum/3
19     return !obsolete in f90 & f95
20 end function cal_avg
```

Calling function

Arguments

Called function **type**
definition

Called function **definition**

Type declaration of the
arguments, and variables

return is necessary in earlier
Fortran, but now is obsolete

Function name **cal_avg** is
used in an assignment
statement

end is mandatory to
complete the function
definition

Functions: Internal Definition

- Calculation of cube-root of any given number using **internal function** definition

```
*main.f95 x
1  ! A fortran95 program for G95
2  ! For CSE 425, Fortran Basic Tutorial
3  ! Function definition: Internal, using the CONTAINS
4  program main
5      implicit none
6      real x, r
7      write(*,*) 'Enter your desired cube-root finding:'
8      read*, x
9      r = cube_root(x)
10     write(*,*) 'Cube-root of', x, 'is', r
11
12     contains
13
14     real function cube_root(x)
15     implicit none
16     real x
17     intent(in) x
18     cube_root = exp(log(x)/3.0)
19     end function cube_root
20
21 end program main
```

Internal function
definition

Without Intent (in)

- Calculation of cube-root of any given number using **internal function** definition

```
main.f95 x
1  ! A fortran95 program for G95
2  ! For CSE 425, Fortran Basic Tutorial
3  ! Function definition: Internal, using the CONTAINS c
4  program main
5      implicit none
6      real x, r
7      write(*,*) 'Enter your desired cube-root finding:'
8      read*, x
9      r = cube_root(x)
10     write(*,*) 'Cube-root of', x, 'is', r
11
12     contains
13
14     real function cube_root(x)
15     implicit none
16     real x
17     !intent(in) x
18     x = (log(x)/3.0)
19     cube_root = exp(x)
20     end function cube_root
21
22 end program main
23
```

Intent (in) commented out

What is the value of X printed after cube-root calculation?

Subroutines

- Subroutines are similar to external functions defined in Fortran, but with an exception that they do not return value
- Instead, subroutines can modify the arguments used to call it by the program
- Swapping numbers

```
function name(arg1, arg2 ...)  
[declarations, including those  
for the arguments]  
[executable statements]  
end function name
```

```
subroutine name(arg1, arg2,)  
[declarations,]  
[executable statements]  
end subroutine name
```

Subroutines: Example

- Swapping numbers

```
main.f95  X
1      ! A fortran95 program for G95
2      ! Subroutine example
3      program swap_number_main
4      implicit none
5      real :: num1, num2
6      print *, 'Enter two numbers:'
7      ! Read in two values
8      read(*,*) num1, num2
9      call swap_num(num1,num2)
10     write(*,*) num1,num2
11     contains ! syntax to include subroutine
12     subroutine swap_num(x_first, y_second)
13     real :: x_first, y_second, temp
14     temp = x_first
15     x_first = y_second
16     y_second = temp
17     end subroutine swap_num
18     end program swap_number_main
```

Fortran code: Newton's Method

- Root finding method

Example: $f(x) = x^3 + x - 3$ $f'(x) = 3x^2 + 1$

Newton's Method:

$$f(x) = 0 \Rightarrow x = \text{root} \quad x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

where $f'(x_i)$ is the first derivative calculated at x_i

```
there  X  main.f95  X
1      ! A fortran95 program for G95
2      ! Newton's Method: Solve  $f(x) = 0$ 
3      !  $f(x) = x^3 + x - 3$ ,  $df(x) = 3x^2 + 1$ 
4      program rootfinding_newton
5          implicit none ! define all variables
6          real :: x = 1
7          real :: error_tol = 0.000001
8          integer i ! no colon means we cannot assign value
9          integer maxit
10         integer converg
11
12         i = 0 ! assigning value
13         maxit = 30
14         converg = 0
15         do while (converg == 0 .and. i < maxit)
16             x = x - f_x(x)/f_deriv_x(x)
17             write(*,*) x, f_x(x)
18             i = i+1
19             if(abs(f_x(x))<=error_tol) converg = 1
20         end do
21
22         if (converg == 1) then
23             write (*,*) 'The method converged'
24         else
25             write(*,*) 'The method did not converge'
26         end if
```

Conditional do loop

contains: ends the main program; functions and subroutines are defined right after this.

```
28     contains !end of main, starts functions & subroutines
29     function f_x(x)
30         real f_x, x
31         f_x = x**3 + x - 3
32     end function f_x
33
34     function f_deriv_x(x)
35         real f_deriv_x, x
36         f_deriv_x = 3*x**2 + 1
37     end function f_deriv_x
38 end program rootfinding_newton
```

Data Generation and Storing

```
4  ! read and visualized using gnuplot.
5  program plot_example
6  implicit none
7  integer i, n
8  real :: pi, area, radius
9  real, dimension(:), allocatable :: radius_array
10 real, dimension(:), allocatable :: area_array
11 pi=3.1416
12
13 write(*,*) 'Enter the data points:!'
14 read *,n
15
16 allocate(radius_array(n))
17 allocate(area_array(n))
18 do i = 1,n
19     radius=i
20     area=pi*radius**2
21     radius_array(i) = radius
22     area_array(i) = area
23 end do
24
25 ! Writing output to a file
26
27 open(12, file = 'example.dat', status = 'new')
28
29 do i = 1,n
30     write(12,*) radius_array(i),area_array(i)
31 end do
32
33 close(12)
34
35 deallocate(radius_array)
36 deallocate(area_array)
37
38 end program plot_example
39
```

Dynamic array, based on user input

Allocating memory storage

Open command opens a file with integer id 12

.txt, .dat

Write (12, *) writes data to the file

Close (12) closes the file opened

Data Plot

- We use Gnuplot for data visualization from .dat or .txt file data generated in Fortran for a simple example.

```
GNUPLOT
Version 5.4 patchlevel 8    last modified 2023-06-01

Copyright (C) 1986-1993, 1998, 2004, 2007-2023
Thomas Williams, Colin Kelley and many others

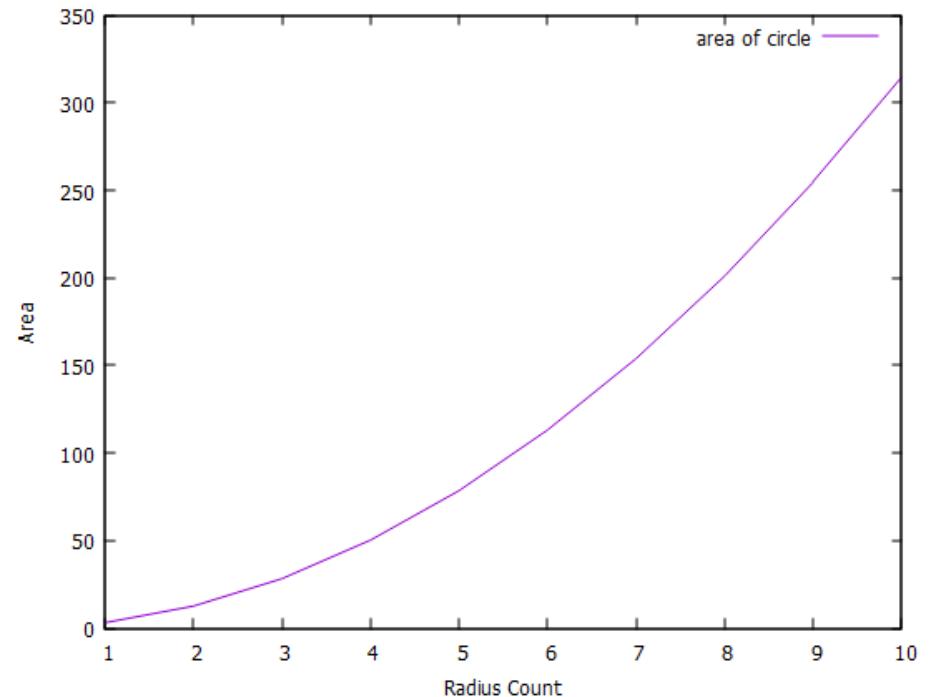
gnuplot home:      http://www.gnuplot.info
faq, bugs, etc:    type "help FAQ"
immediate help:    type "help" (plot window: hit 'h')

Terminal type is now 'qt'
gnuplot> pwd
C:\Users\User\Documents
gnuplot> cd FORTRAN_PRACTICE
      ^
      expecting directory name

gnuplot> cd "FORTRAN_PRACTICE"
gnuplot> pwd
C:\Users\User\Documents\FORTRAN_PRACTICE
gnuplot> cd FORTRAN_PLOT
      ^
      expecting directory name

gnuplot> cd "FORTRAN_PLOT"
gnuplot> pwd
C:\Users\User\Documents\FORTRAN_PRACTICE\FORTRAN_PLOT
gnuplot> plot 'example.dat'
gnuplot> plot 'example.dat' using 1:2 with lines title "area of circle"
gnuplot> set xlabel "Radius Count"
gnuplot> set xlabel Radius Count
      ^
      unexpected or unrecognized token: Radius

gnuplot> set xlabel 'Radius Count'
gnuplot> set xlabel "Area"
gnuplot> set xlabel 'Radius Count'
gnuplot> set ylabel "Area"
gnuplot> pwd
C:\Users\User\Documents\FORTRAN_PRACTICE\FORTRAN_PLOT
gnuplot> _
```



References

- [1]. <http://www.chem.ox.ac.uk/fortran/>
- [2]. Programming in Fortran 95, Computational Physics, University of Cambridge
- [3]. <https://web.stanford.edu/class/me200c>
- [4]. Computing with Fortran, Institute of Energy Technology, ETH, Zürich